
Introduction to Database

XXXXXX

Project

Deadline: Monday 09/08/2026 @ 23:59

[Total Mark is 20]

Student Details:

CRN:50242

Name: Jana Yasir Almadhun

ID: S240031903

Name: Madaen Saleh Alshuqairan

ID: S240006334

Name: Reem Khalid Alshehri

ID: S240065623

Name: Riman Mohammed Alhazme

ID: S240039097

Instructions:

- You must submit two separate copies (**one Word file and one PDF file**) using the Assignment Template on Blackboard via the allocated folder. These files **must not be in compressed format**.
- It is your responsibility to check and make sure that you have uploaded both the correct files.
- Zero mark will be given if you try to bypass the SafeAssign (e.g. misspell words, remove spaces between words, hide characters, use different character sets, **convert text into image** or languages other than English or any kind of manipulation).
- Email submission will not be accepted.
- You are advised to make your work clear and well-presented. This includes filling your information on the cover page.
- You must use this template, failing which will result in zero mark.
- You **MUST** show all your work, and text must not be converted into an image, unless specified otherwise by the question.
- Late submission will result in ZERO mark.
- The work should be your own, copying from students or other resources will result in ZERO mark.
- Use **Times New Roman** font for all your answers.

Project Instructions

- You can work on this project as a group (minimum 3 and maximum 4 students). Each group member must submit the project individually with all group member names mentioned on the cover page.
- This project **worth 20 marks** and will be distributed as in the following:
 - Database Design. (6 marks)
 - Database Implementation. (6 marks)
 - SQL Operations. (6 marks)
 - Project Reflection. (2 marks)
- Each leader in the group must submit one report about their chosen Project via the Blackboard (**Email submission will not be accepted and will be awarded ZERO marks**) containing the following:
 - A. Task 1 : database design
 - B. Task 2: database implementation
 - C. All requested queries/results
 - D. Screenshots from MySQL (or any other software you use) of **all the tables** after population and **query results**.
 - E. Reflection (300–500 words) describing the team's experience, challenges, design decisions, and AI usage (if applicable).
- Submit one **SQL file** containing the complete database implementation, including database creation, tables, data insertion, required queries, VIEW, and TRIGGER.
- Submit a **GitHub** (or GitLab) repository containing the complete project with regular commits and meaningful commit messages.
- Submit a **progress report** describing the completed work, key decisions, challenges, future plan, and contribution of each group member.
- Include **links** to the live report document, GitHub/GitLab repository, and Google Colab notebook (if applicable) in the **Project Artifacts** section of the final report.
- You are advised to make your work clear and well presented; marks may be reduced for poor presentation. This includes filling in your information on the cover page.
- You MUST show all your work, and text **must not** be converted into an image unless specified otherwise by the question.
- **Late submission will result in ZERO marks being awarded.**
- The work should be your own. Copying **from students or other resources, including AI software, will result in ZERO marks.**

*Learning**Outcome(s):**CLO1, CLO2, CLO3,**CLO4, CLO5*

Project Description

Design and Implementation of a Smart Clinic Database System

A private clinic currently manages its patient information manually, making it difficult to organize appointments, treatments, and payments. The clinic has decided to develop a database system to improve data organization and retrieval.

Your team is required to design, implement, and test a complete relational database using the concepts learned throughout this course. The project should demonstrate your understanding of database design, ER/EER modeling, relational mapping, SQL implementation, and advanced SQL queries. The project must be completed using MySQL.

Mid Project

1. Work Completed During This Period

During this phase, our team analyzed the clinic requirements and identified the main entities of the database, including Patients, Doctors, Appointments, Treatments, Medicines, and Payments, we designed the ER/EER model, defined the relationships between entities, assigned primary and foreign keys, created the relational schema, we also started implementing the database in MySQL by creating the tables with the required constraints and inserting sample records.

2. One Key Decision Made

One important design decision was to separate Appointments and Treatments into two related entities instead of storing all treatment information inside the appointment table, this follows the database design principles discussed in the course by reducing redundancy and improving data integrity.

3. One Problem Encountered

The main challenge was identifying the correct cardinality and foreign key relationships between patients, appointments, treatments, and payments, after reviewing the ER/EER model and testing the schema, we modified the relationships to ensure consistency and avoid data anomalies.

4. Plan for the Next Phase

Our next step is to complete the database implementation by inserting the remaining records, writing the required SQL queries, including SELECT, JOIN, Nested Queries, GROUP BY, UPDATE, and DELETE, creating the required VIEW and TRIGGER, finally, we will test the database and prepare the final report.

5. Team Members' Contributions

Reem Alshehri : Designed the ER/EER diagram and relational schema

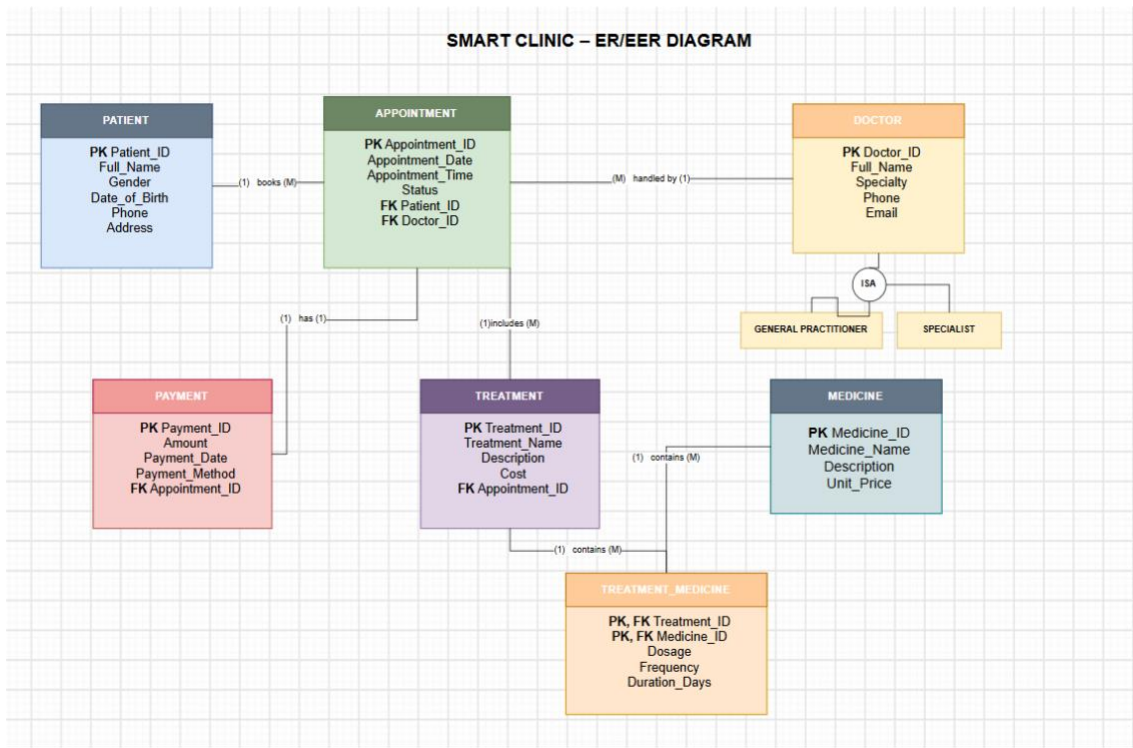
Madaen Alshuqairan: Implemented the database tables in MySQL

Jana Almadhun: Inserted sample data and developed SQL queries

Riman Alhazme: Tested the database and prepared the documentation.

Task 1 – Database Design (6 Marks)

- Design a complete database for the clinic that includes at least six entities such as Patients, Doctors, Appointments, Treatments, Medicines, Payments, or any other appropriate entities.
- A complete ER/EER diagram.
- Primary and foreign keys.
- Appropriate relationships with cardinality.
- At least one specialization/generalization (EER feature).
- Any assumptions made during the design process.



LIST OF ENTITIES

Entity	Description	Primary Key (PK)	Main Attributes	Foreign Keys (FK)	Relationships
Patient	Stores patient information.	Patient_ID	Full_Name, Gender, Date_of_Birth, Phone, Address	-	One Patient books many Appointments.
Appointment	Stores appointment details.	Appointment_ID	Appointment_Date, Appointment_Time, Status	Patient_ID, Doctor_ID	Many Appointments belong to one Patient and one Doctor. One Appointment has one Payment and includes many Treatments.
Doctor	Stores doctor information.	Doctor_ID	Full_Name, Specialty, Phone, Email	-	One Doctor handles many Appointments. A Doctor is either a General Practitioner or a Specialist (not both).
Payment	Stores payment information.	Payment_ID	Amount, Payment_Date, Payment_Method	Appointment_ID	One Appointment has one Payment.
Treatment	Stores treatment information.	Treatment_ID	Treatment_Name, Description, Cost	Appointment_ID	One Appointment includes many Treatments. Many Treatments can use many Medicines.
Medicine	Stores medicine information.	Medicine_ID	Medicine_Name, Description, Unit_Price	-	Many Medicines can be used in many Treatments.
Treatment_Medicine	Stores the relationship between Treatments and Medicines.	Treatment_ID, Medicine_ID (Composite PK)	Dosage, Frequency, Duration_Days	Treatment_ID, Medicine_ID	Resolves the many-to-many relationship between Treatment and Medicine.

Design Assumptions:

- A patient can book multiple Appointments, but each Appointment belongs to only one patient.
- A Doctor can handle multiple Appointments, but each Appointment is handled by one Doctor.
- Each Appointment can include multiple Treatments, and can have one payment record.
- A Doctor is generalized into two specializations: General Practitioner or Specialist.
- Treatments and Medicines have a Many-to-Many relationship, resolved using the Treatment_Medicine associative entity.

Task 2 – Database Implementation (6 Marks)

Complete the following tasks using MySQL. Include screenshots of both your SQL code and the corresponding output (results) for each task:

- Create the database and all required tables.
- Use appropriate data types and constraints (PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, etc.).
- Insert at least 5 records into each main table.

```
CREATE DATABASE Smart_Clinic;
USE Smart_Clinic;

CREATE TABLE Patient (
    Patient_ID INT PRIMARY KEY,
    Full_Name VARCHAR(100) NOT NULL,
    Gender VARCHAR(10),
    Date_of_Birth DATE,
    Phone VARCHAR(20),
    Address VARCHAR(255)
);

CREATE TABLE Doctor (
    Doctor_ID INT PRIMARY KEY,
    Full_Name VARCHAR(100) NOT NULL,
    Specialty VARCHAR(100) NOT NULL,
    Phone VARCHAR(20),
    Email VARCHAR(100) UNIQUE
);

CREATE TABLE Appointment (
    Appointment_ID INT PRIMARY KEY,
    Appointment_Date DATE NOT NULL,
    Appointment_Time TIME NOT NULL,
    Status VARCHAR(50),
    Patient_ID INT,
    Doctor_ID INT,

    FOREIGN KEY (Patient_ID) REFERENCES Patient(Patient_ID),
    FOREIGN KEY (Doctor_ID) REFERENCES Doctor(Doctor_ID)
);

CREATE TABLE Payment (
    Payment_ID INT PRIMARY KEY,
    Amount DECIMAL(10,2),
    Payment_Date DATE,
    Payment_Method VARCHAR(50),
    Appointment_ID INT UNIQUE,

    FOREIGN KEY (Appointment_ID)
        REFERENCES Appointment(Appointment_ID)
);

CREATE TABLE Treatment (
    Treatment_ID INT PRIMARY KEY,
    Treatment_Name VARCHAR(100) NOT NULL,
    Description TEXT,
    Appointment_ID INT,

    FOREIGN KEY (Appointment_ID)
        REFERENCES Appointment(Appointment_ID)
);

CREATE TABLE Medicine (
    Medicine_ID INT PRIMARY KEY,
    Medicine_Name VARCHAR(100) NOT NULL,
    Description TEXT,
    Unit_Price DECIMAL(10,2)
);

CREATE TABLE Treatment_Medicine (
    Treatment_ID INT,
    Medicine_ID INT,
```

```

Quantity INT,

PRIMARY KEY (Treatment_ID, Medicine_ID),

FOREIGN KEY (Treatment_ID)
REFERENCES Treatment(Treatment_ID),

FOREIGN KEY (Medicine_ID)
REFERENCES Medicine(Medicine_ID)
);

INSERT INTO Patient
(Patient_ID, Full_Name, Gender, Date_of_Birth, Phone, Address)
VALUES
(1, 'Sara Ahmed', 'Female', '2001-05-14', '0501234567', 'Riyadh'),
(2, 'Noura Ali', 'Female', '1998-11-02', '0502345678', 'Jeddah'),
(3, 'Maha Salem', 'Female', '2003-03-21', '0503456789', 'Dammam'),
(4, 'Omar Khalid', 'Male', '1995-08-10', '0504567890', 'Riyadh'),
(5, 'Fahad Mohammed', 'Male', '2000-12-30', '0505678901', 'Makkah');

INSERT INTO Doctor
(Doctor_ID, Full_Name, Specialty, Phone, Email)
VALUES
(1, 'Dr. Ahmed Hassan', 'General Practitioner', '0551111111', 'ahmed@clinic.com'),
(2, 'Dr. Lina Omar', 'Dentist', '0552222222', 'lina@clinic.com'),
(3, 'Dr. Khalid Ali', 'Cardiologist', '0553333333', 'khalid@clinic.com'),
(4, 'Dr. Sara Mohammed', 'Dermatologist', '0554444444', 'sara@clinic.com'),
(5, 'Dr. Faisal Abdullah', 'Pediatrician', '0555555555', 'faisal@clinic.com');

INSERT INTO Appointment
(Appointment_ID, Appointment_Date, Appointment_Time, Status, Patient_ID, Doctor_ID)
VALUES
(1, '2025-11-01', '09:00:00', 'Completed', 1, 1),
(2, '2025-11-02', '10:30:00', 'Completed', 2, 2),
(3, '2025-11-03', '11:00:00', 'Scheduled', 3, 3),
(4, '2025-11-04', '01:00:00', 'Completed', 4, 4),
(5, '2025-11-05', '03:30:00', 'Scheduled', 5, 5);

INSERT INTO Payment
(Payment_ID, Amount, Payment_Date, Payment_Method, Appointment_ID)
VALUES
(1, 200.00, '2025-11-01', 'Cash', 1),
(2, 350.00, '2025-11-02', 'Card', 2),
(3, 500.00, '2025-11-03', 'Cash', 3),
(4, 250.00, '2025-11-04', 'Card', 4),
(5, 300.00, '2025-11-05', 'Cash', 5);

INSERT INTO Treatment
(Treatment_ID, Treatment_Name, Description, Appointment_ID)
VALUES
(1, 'General Checkup', 'Routine examination', 1),
(2, 'Teeth Cleaning', 'Dental cleaning', 2),
(3, 'Heart Check', 'ECG examination', 3),
(4, 'Skin Treatment', 'Acne treatment', 4),
(5, 'Child Consultation', 'Routine pediatric visit', 5);

INSERT INTO Medicine
(Medicine_ID, Medicine_Name, Description, Unit_Price)
VALUES
(1, 'Paracetamol', 'Pain relief', 15.00),
(2, 'Amoxicillin', 'Antibiotic', 35.00),
(3, 'Ibuprofen', 'Anti-inflammatory', 20.00),
(4, 'Vitamin C', 'Supplement', 25.00),

```

```
(5, 'Cough Syrup', 'Cough medicine', 18.00);
```

```
INSERT INTO Treatment_Medicine  
(Treatment_ID, Medicine_ID, Quantity)  
VALUES  
(1, 1, 2),  
(2, 2, 1),  
(3, 3, 2),  
(4, 4, 1),  
(5, 5, 1);
```

```
SELECT * FROM Patient;  
SELECT * FROM Doctor;  
SELECT * FROM Appointment;  
SELECT * FROM Payment;  
SELECT * FROM Treatment;  
SELECT * FROM Medicine;  
SELECT * FROM Treatment_Medicine
```

The screenshots show the MySQL Workbench interface with the following data:

Patients Table:

Patient_ID	Pt_Name	Gender	Date_of_Birth	Phone	Address
1	Sara Ahmed	Female	2001-05-14	081234567	Riyadh
2	Noura Al	Female	1998-11-02	090245678	Jeddah
3	Maha Salem	Female	2003-03-21	050456789	Dammam
4	Omar Khalid	Male	1985-08-10	050456789	Riyadh
5	Fahad Mohammed	Male	2000-12-30	0505678901	Makkah

Doctors Table:

Doctor_ID	Pt_Name	Specialty	Phone	Email
1	Dr. Ahmed Hassan	General Practitioner	0551111111	ahmed@drmc.com
2	Dr. Lina Omar	Dermatologist	0552222222	lina@drmc.com
3	Dr. Khalid Ali	Cardiologist	0553333333	khalid@drmc.com
4	Dr. Sara Mohammed	Dermatologist	0554444444	sara@drmc.com
5	Dr. Faisal Abdulrah	Pediatrician	0555555555	faisal@drmc.com

Appointments Table:

Appointment_ID	Appointment_Date	Appointment_Time	Status	Patient_ID	Doctor_ID
1	2025-11-01	09:00:00	Completed	1	1
2	2025-11-02	10:30:00	Completed	2	2
3	2025-11-03	11:00:00	Scheduled	3	3
4	2025-11-04	01:00:00	Completed	4	4
5	2025-11-05	03:30:00	Scheduled	5	5

Payments Table:

Payment_ID	Amount	Payment_Date	Payment_Method	Appointment_ID
1	200.00	2025-11-01	Cash	1
2	350.00	2025-11-02	Card	2
3	900.00	2025-11-03	Cash	3
4	250.00	2025-11-04	Card	4
5	300.00	2025-11-05	Cash	5

Action Output Log:

Time	Action	Message	Duration / Fech
19 15:51:20	SELECT * FROM Doctor LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
20 15:51:20	SELECT * FROM Appointment LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
21 15:51:20	SELECT * FROM Payment LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
22 15:51:20	SELECT * FROM Treatment LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
23 15:51:20	SELECT * FROM Medicine LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
24 15:51:20	SELECT * FROM Treatment_Medicine LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec

MySQL Workbench interface showing a query result for Patient appointments. The query is:

```
SELECT * FROM Patient;
SELECT * FROM Doctor;
SELECT * FROM Appointment;
SELECT * FROM Payment;
SELECT * FROM Treatment;
SELECT * FROM Medicine;
SELECT * FROM Treatment_Medicine;
```

The result grid displays the following data:

Treatment_ID	Treatment_Name	Description	Appointment_ID
1	General Checkup	Routine examination	1
2	Teeth Cleaning	Dental cleaning	2
3	Heart Check	ECG examination	3
4	Skin Treatment	Acne treatment	4
5	Child Consultation	Routine pediatric visit	5

The Action Output pane shows the execution of the query, indicating that 5 rows were returned for each table.

MySQL Workbench interface showing a query result for Medicine details. The query is:

```
SELECT * FROM Patient;
SELECT * FROM Doctor;
SELECT * FROM Appointment;
SELECT * FROM Payment;
SELECT * FROM Treatment;
SELECT * FROM Medicine;
SELECT * FROM Treatment_Medicine;
```

The result grid displays the following data:

Medicine_ID	Medicine_Name	Description	Unit_Price
1	Paracetamol	Pain relief	15.00
2	Amoxicillin	Antibiotic	35.00
3	Ibuprofen	Anti-inflammatory	20.00
4	Vitamin C	Supplement	25.00
5	Cough Syrup	Cough medicine	18.00

The Action Output pane shows the execution of the query, indicating that 5 rows were returned for each table.

MySQL Workbench interface showing a query result for Treatment_Medicine details. The query is:

```
SELECT * FROM Patient;
SELECT * FROM Doctor;
SELECT * FROM Appointment;
SELECT * FROM Payment;
SELECT * FROM Treatment;
SELECT * FROM Medicine;
SELECT * FROM Treatment_Medicine;
```

The result grid displays the following data:

Treatment_ID	Medicine_ID	Quantity
1	1	2
2	2	1
3	3	2
4	4	1
5	5	1

The Action Output pane shows the execution of the query, indicating that 5 rows were returned for each table.

Task 3 – SQL Operations (6 Marks)

Complete the following tasks using MySQL. For each task, include screenshots of both your SQL code and the corresponding output (results), along with a brief description (one to two sentences) explaining the purpose of the query and the information it is intended to retrieve.

- Write SELECT statements.
- Write JOIN queries.
- Write nested queries.
- Use aggregate functions with GROUP BY.
- Write UPDATE and DELETE statements.
- Create one VIEW.
- Create one TRIGGER.

SELECT

Description: Retrieves all patient information from the Patient table.

```

149  -- -----
150
151  -- SELECT
152  ●  USE Smart_Clinic;
153  ●  SELECT *
154      FROM Patient;
155

```

Patient_ID	Full_Name	Gender	Date_of_Birth	Phone	Address
1	Sara Ahmed	Female	2001-05-14	0500000000	Riyadh
2	Noura Ali	Female	1998-11-02	0502345678	Jeddah
3	Maha Salem	Female	2003-03-21	0503456789	Dammam
4	Omar Khalid	Male	1995-08-10	0504567890	Riyadh
5	Fahad Mohammed	Male	2000-12-30	0505678901	Makkah
NULL	NULL	NULL	NULL	NULL	NULL

JOIN Query

Description: Retrieves appointment information by combining related data from the Appointment, Patient, and Doctor tables.

```
157  -- JOIN
158  SELECT
159      p.Full_Name AS Patient,
160      d.Full_Name AS Doctor,
161      a.Appointment_Date,
162      a.Status
163  FROM Appointment a
164  JOIN Patient p ON a.Patient_ID = p.Patient_ID
165  JOIN Doctor d ON a.Doctor_ID = d.Doctor_ID;
166
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	Patient	Doctor	Appointment_Date	Status
▶	Sara Ahmed	Dr. Ahmed Hassan	2025-11-01	Completed
	Noura Ali	Dr. Lina Omar	2025-11-02	Completed
	Maha Salem	Dr. Khalid Ali	2025-11-03	Scheduled
	Omar Khalid	Dr. Sara Mohammed	2025-11-04	Completed
	Fahad Mohammed	Dr. Faisal Abdullah	2025-11-05	Scheduled

Nested Query

Description: Retrieves the names of patients who have completed appointments using a nested query.

```
167
168  -- Nested Query
169  •  SELECT Full_Name
170     FROM Patient
171     WHERE Patient_ID IN (
172         SELECT Patient_ID
173         FROM Appointment
174         WHERE Status = 'Completed'
175     );
176
```

Result Grid

	Full_Name
▶	Sara Ahmed
	Noura Ali
	Omar Khalid

Filter Rows: Export: Wrap Cell Content:

Aggregate Function with GROUP BY

Description: Counts the number of appointments for each appointment status using the COUNT function and GROUP BY clause.

```
177
178      -- GROUP BY
179 •   SELECT Status, COUNT(*) AS Total_Appointments
180      FROM Appointment
181      GROUP BY Status;
182
183
184      -- UPDATE
185 •   UPDATE Appointment
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	Status	Total_Appointments			
▶	Completed	3			
	Scheduled	2			

UPDATE Statement

Description: Updates the phone number of the patient with Patient ID 1.

```
183
184 -- UPDATE
185 • UPDATE Appointment
186 SET Status = 'Completed'
187 WHERE Appointment_ID = 3;
188 • SELECT *
189 FROM Appointment
190 WHERE Appointment_ID = 3;
```

Result Grid						
Filter Rows:						
Edit:   						
Export/Import:  						
Wrap Cell Content: 						
	Appointment_ID	Appointment_Date	Appointment_Time	Status	Patient_ID	Doctor_ID
▶	3	2025-11-03	11:00:00	Completed	3	3
•	NULL	NULL	NULL	NULL	NULL	NULL

DELETE Statement

Description: Deletes the treatment-medicine record with Treatment ID 5 from the Treatment_Medicine table.

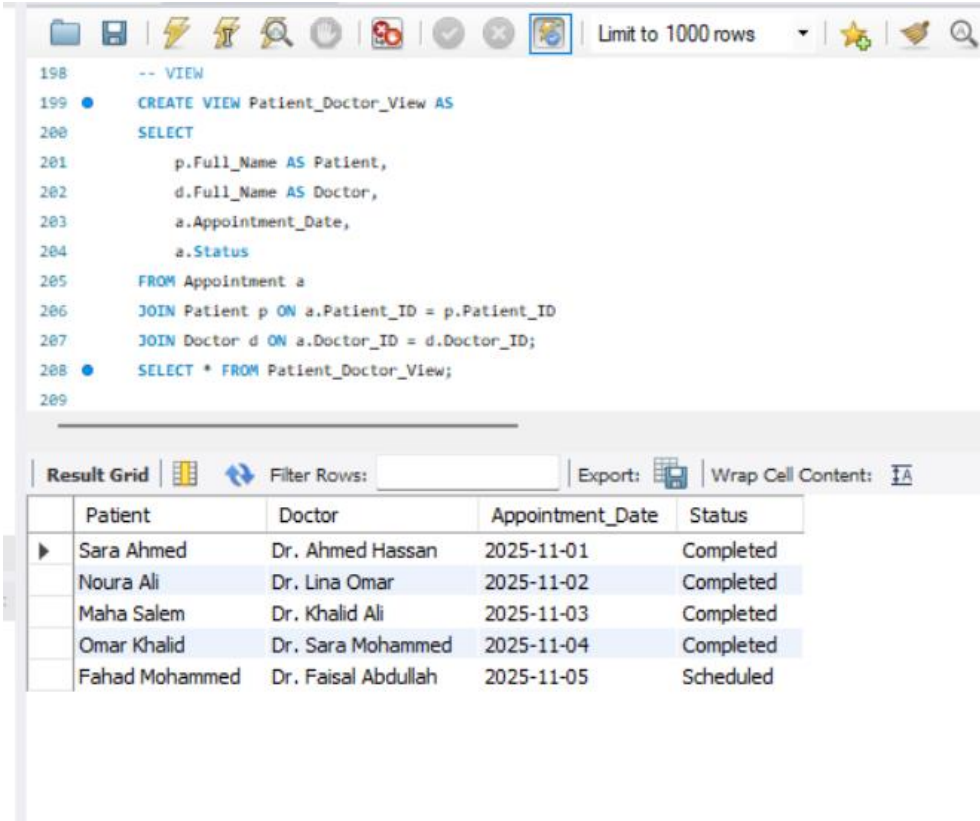
```
187
188      -- DELETE
189  ●   DELETE FROM Treatment_Medicine
190      WHERE Treatment_ID = 5;
191  ●   SELECT * FROM Treatment_Medicine;
192
193      -- VIEW
```

Result Grid | Filter Rows: | Edit:

	Treatment_ID	Medicine_ID	Quantity
▶	1	1	2
	2	2	1
	3	3	2
	4	4	1
•	NULL	NULL	NULL

VIEW

Description: Creates a virtual table that displays patient names, doctor names, appointment dates, and appointment status.



```
-- VIEW
199 CREATE VIEW Patient_Doctor_View AS
200 SELECT
201     p.Full_Name AS Patient,
202     d.Full_Name AS Doctor,
203     a.Appointment_Date,
204     a.Status
205 FROM Appointment a
206 JOIN Patient p ON a.Patient_ID = p.Patient_ID
207 JOIN Doctor d ON a.Doctor_ID = d.Doctor_ID;
208 SELECT * FROM Patient_Doctor_View;
209
```

	Patient	Doctor	Appointment_Date	Status
▶	Sara Ahmed	Dr. Ahmed Hassan	2025-11-01	Completed
	Noura Ali	Dr. Lina Omar	2025-11-02	Completed
	Maha Salem	Dr. Khalid Ali	2025-11-03	Completed
	Omar Khalid	Dr. Sara Mohammed	2025-11-04	Completed
	Fahad Mohammed	Dr. Faisal Abdullah	2025-11-05	Scheduled

TRIGGER

Description: Sets the status of a new appointment to 'Scheduled' if no status is provided.

```
210 -- TRIGGER
211 DELIMITER $$
212
213 CREATE TRIGGER Check_Appointment_Status
214 BEFORE INSERT ON Appointment
215 FOR EACH ROW
216 BEGIN
217     IF NEW.Status IS NULL THEN
218         SET NEW.Status = 'Scheduled';
219     END IF;
220 END$$
221
222 DELIMITER ;
```

Result Grid

	Appointment_ID	Appointment_Date	Appointment_Time	Status	Patient_ID	Doctor_ID
▶	6	2025-11-06	12:00:00	Scheduled	1	1
•	NULL	NULL	NULL	NULL	NULL	NULL

```
216 BEGIN
217     IF NEW.Status IS NULL THEN
218         SET NEW.Status = 'Scheduled';
219     END IF;
220 END$$
221
222 DELIMITER ;
223 INSERT INTO Appointment
224 (Appointment_ID, Appointment_Date, Appointment_Time, Patient_ID, Doctor_ID, Status)
225 VALUES
226 (6, '2025-11-06', '12:00:00', 1, 1, NULL);
227 SELECT *
228 FROM Appointment
229 WHERE Appointment_ID = 6;
```

Result Grid

	Appointment_ID	Appointment_Date	Appointment_Time	Status	Patient_ID	Doctor_ID
▶	6	2025-11-06	12:00:00	Scheduled	1	1
•	NULL	NULL	NULL	NULL	NULL	NULL

Task 4 – Reflection (2 Marks)

Write a **300–500-word reflection** on your experience while completing this project by answering the following questions. Your responses should be clear, concise, and supported with specific examples from your own work.

1. Describe one specific challenge or obstacle you encountered during this project and explain in detail how you identified and resolved it.
2. Explain why you chose your specific approach/method/model over at least one alternative you considered, and what tradeoffs informed that decision.
3. If you used any AI tools during this project (e.g., ChatGPT, Claude, Copilot), disclose which tool(s), for what specific purpose (e.g., debugging, grammar checking, brainstorming), and how you verified or modified the output.
4. What would you do differently if you were to redo this project with more time or resources?

Question 1

One of the main challenges we encountered was designing the relationships between Patients, Appointments, Treatments, and Payments while maintaining data integrity, at first we were unsure whether treatment information should be stored directly in the appointment table or in a separate table, after reviewing the ER/EER design principles and testing different relationships, we decided to create a separate Treatments entity linked to Appointments through foreign keys, this solution reduced data redundancy, improved flexibility, and maintained a normalized database structure.

Question 2

We chose a normalized relational database design because it minimizes data redundancy, improves data consistency, and maintains referential integrity through primary and foreign keys, the alternative approach was to combine several pieces of information into fewer tables to simplify the design, however, that approach would increase duplicate data and make future updates more difficult, although the

normalized design required more relationships and tables, it provides better scalability and follows the database design principles taught in the course.

Question 3

We used ChatGPT only for grammar checking and correcting minor language mistakes in the report, After reviewing the corrections, we ensured that the final text reflected our own work before including it in the report.

Question 4

If we had more time, we would improve the project by developing a graphical user interface connected to the database, adding more realistic datasets, and performing more comprehensive testing under different scenarios, we would also enhance the database by adding additional features such as appointment reminders, user authentication, and advanced reports, these improvements would make the system more practical and closer to a real clinic management system.

Deliverables

1. **Project Report (PDF)** – System description, ER/EER diagram, relational schema, SQL code, screenshots, and reflection.
2. **SQL Script (.sql)** – Complete SQL implementation of the project.
3. **GitHub Repository** – Repository link with the complete project and commit history.
4. **Mid-Project Progress Report** – Summary of progress, challenges, decisions, and team contributions.
5. **Project Artifacts**
 - Live report document (with edit history): [link]
 - GitHub/GitLab repository (with commit history): [link]
 - Google Colab notebook (if applicable): [link]